ABHISEK GUPTA (2402MC03) MA3105: ASSIGNMENT_08

---

1. Virtual Address to Physical Address Translation:

System Call: uint64 va2pa(uint64 va)

Mechanism: Walks the process's 3-level SV39 page directory using walk(pagetable, va, 0).

Validation: Rejects invalid virtual addresses (va >= MAXVA), unmapped pages, and kernel-only pages ((pte & PTE_U) == 0) by returning -1.

Translation Output: Formats and prints the Level 2/1/0 indices (PX(2, va), PX(1, va), PX(0, va)), page offset (va & 0xFFF), and flags (P, W, U).

Result: Successfully confirmed address translations for code/global data, user stack, dynamic heap (malloc), unmapped bounds, and demonstrated that parent and child share identical virtual addresses but map to distinct physical frames after fork().

---

2. Physical Memory Allocation Statistics:

System Call: int getmeminfo(struct meminfo *info)

Kernel Tracking:
total_pages: Total frames initialized during boot (PHYSTOP - KERNBASE) / PGSIZE = 32735.
free_pages: Tracked via spinlock-protected atomic counters in kalloc() and kfree().
user_pages: Evaluated as p->sz / PGSIZE.
pagetable_pages: Recursively calculated by counting valid intermediate page table pages in the SV39 tree.

Result: Demonstrated exact tracking: sbrk(+10 pages) reduced free pages by 10; child fork() allocated its copy; wait() fully reclaimed frames, confirming zero memory leaks; shrinking returned all frames to the free list.

---

3. Dumping Process Page Tables:

System Call: int vmprint(void)

Mechanism: Traverses the root L2 page directory, L1 directories, and L0 page tables.

Output: Uses structured indentation ([Page Directory L2 idx ...], [Page Directory L1 idx ...], [PTE ...]), prints virtual-to-physical mappings with permission flags, explicitly flags the stack guard page (PTE without PTE_U), and provides user vs kernel mapping summaries.

Result: Accurately dumped page tables across process lifecycle states: baseline process startup, after heap expansion (sbrk(+8 pages)), and after heap reduction (sbrk(-1 page)).

---

4. Demand Paging:

Lazy sbrk(): sys_sbrk(n) for n > 0 merely increases myproc()->sz without calling growproc(). For n < 0, it invokes growproc(n) to immediately free physical frames via uvmunmap().

Page Fault Handling: Catches load (scause 13) and store (scause 15) page faults in usertrap(), calls vmfault(), allocates a zeroed frame with kalloc(), and maps it with PTE_R | PTE_W | PTE_U.

Edge Cases Handled:
Rejects faults with va >= p->sz or below user stack (va < p->trapframe->sp).
Gracefully terminates processes on physical memory exhaustion.
Transparently handles unmapped lazy buffers passed to kernel system calls in copyin() and copyout().
In uvmcopy(), unallocated lazy pages are skipped during fork().

Result: 32 MB allocated with 0 physical frames consumed; touching 4 pages allocated exactly 4 physical frames on demand; pipe read into untouched buffer succeeded; invalid access was safely terminated without kernel panic.
